

Brechas del sistema de tracking de remediación frente al marco de referencia

Nota de alcance

Este documento **no es un plan de implementación**. Es un inventario de lo que falta —o está parcialmente resuelto— en el sistema actual de GEF Secure App para que su tracking de remediación de vulnerabilidades cumpla con lo que `Tracking_Remediacion_Documento_Unificado.md` define como un sistema de tracking completo, anclado a NIST, CIS, CISA (VEX, BOD 26-04), SSVC y a la práctica de herramientas de referencia (Dependency-Track, DefectDojo).

Cada punto está ordenado de mayor a menor prioridad (crítico/obligatorio → deseable/avanzado), nombra el concepto que falta, explica por qué importa según el marco, y cita la evidencia concreta encontrada en el código (`archivo:línea`) que sustenta la brecha. Se revisó el backend completo (`backend/src/main/java/com/gef/gefsecureapp/` : modelos, repositorios, servicios y controllers de vulnerabilidades, escaneos, dashboard y GHSA) y el modelo de datos resultante.

Al final se incluye una sección de **fortalezas existentes**, porque varias decisiones de diseño actuales ya están alineadas con el marco y conviene no perderlas de vista al priorizar qué falta.

Nivel de madurez de verificación actual

Según la escala de la Sección 10.1 del marco (0 a 4), el sistema se ubica en un punto **intermedio entre el nivel 1 y el nivel 2**, con un mecanismo que *aparenta* nivel 3 pero no lo alcanza en rigor:

- El cierre real de un caso (`triageStatus = RESUELTA`) lo produce siempre una persona vía `VulnerabilityAuditService.updateStatus()` — es una declaración (nivel 1–2), no una verificación del sistema.
- Existe además un mecanismo automático (`applyLifecycle()`) que marca `detectionStatus = RESOLVED` cuando un CVE deja de aparecer en el escaneo siguiente — el ingrediente central del nivel 3 — pero **no compara versión contra rango vulnerable**, y sobre todo **no verifica cobertura del escaneo antes de inferir la ausencia**. Esto lo degrada: no es una verificación nivel 3 declarada, es una inferencia por ausencia no declarada (ver brecha #2).

Esta ubicación honesta —y no una superior— es el punto de partida correcto para todo lo que sigue.

1. Crítico / obligatorio

Estos puntos son los que el propio marco (Sección 7, "el componente F es el que vuelve auditable a todo el resto") y la Sección 19 (antipatrones) señalan como los que un lector externo o un auditor detecta primero, y sin los cuales no puede sostenerse la afirmación de que existe "tracking" en sentido estricto.

1.1 No existe registro de auditoría inmutable de transiciones (Componente F)

El marco es explícito: sin este componente no hay trazabilidad, no se puede reconstruir el tiempo de permanencia por estado, y las métricas históricas no son recalculables (Sección 12). Es, según el propio documento, el componente que auditablemente sostiene a todos los demás.

En el código, cada cambio de estado **sobrescribe** el registro existente en lugar de generar un evento propio:

- `VulnerabilityAuditService.updateStatus()` (`VulnerabilityAuditService.java:107-126`) aplica el nuevo `triageStatus` directamente sobre la fila de `AssetVulnerability` y llama `resolvedAt = null` cuando el estado deja de ser `RESUELTA` — el timestamp anterior se pierde, no se archiva.
- No existe ninguna entidad `transition` / `history` / `audit_log` en `model/`. La tabla `vulnerabilities_audit` (`VulnerabilityAudit.java`) no cumple ese rol: es un log de *detecciones puntuales de escaneo*, no de *transiciones de estado del caso* — y de hecho una de sus propias filas se muta después de creada (`finding.setAssetVulnerability(av)` en `VulnerabilityAuditService.java:295-296`).
- No existe el campo `actor_type` (HUMANO | ESCANER | POLITICA) que el modelo de referencia (Sección 16) marca como necesario para saber si un cambio lo produjo una persona o un proceso automático.

Esto es exactamente el antipatrón "**Estado sin historia**" de la Sección 19: *"Un único campo de fecha que se sobrescribe en cada transición. [...] Solo se puede calcular un intervalo. La secuencia se pierde."*

1.2 Cierre por ausencia sin verificación de cobertura (Componente A + antipatrón crítico)

La Sección 7.1.1 lo llama *"el error más peligroso de un tracking mal construido"*: interpretar que un hallazgo desapareció porque se remedió, cuando en realidad el escaneo no cubrió ese activo. El efecto es acumulativo y silencioso — las métricas mejoran a medida que empeora la cobertura.

En el código, `VulnerabilityAuditService.applyLifecycle()` (`VulnerabilityAuditService.java:300-306`) marca `RESOLVED` a todo `AssetVulnerability` que estaba `OPEN` en el alcance del escaneo y no fue tocado por la corrida actual — **sin ningún chequeo de cobertura previo**:

- `ScanReport` (`ScanReport.java:58-62`) declara en un comentario que el campo `status` debería distinguir `PENDING/RUNNING/COMPLETED/FAILED/PARTIALLY_COMPLETED`, pero `ScanService.complete()` (`ScanService.java:69-89`) **siempre** setea `"COMPLETED"` de forma incondicional (línea 75) — el estado `PARTIALLY_COMPLETED` no se usa nunca en el código fuente.

- No existen los campos `assets_in_scope` / `assets_analyzed` / `sources_consulted` / `sources_failed` que el modelo de referencia (Sección 7.1.1 y 16) define como mínimos para un registro de ejecución de escaneo.
- Como consecuencia, `applyLifecycle()` no tiene forma de distinguir "se remedió" de "el activo no fue analizado esta vez" — las cinco explicaciones de la Sección 7.1.1 colapsan en una sola inferencia silenciosa.

La Sección 7.1.1 aclara que la inferencia por ausencia es una práctica aceptada (CIS la reconoce) — lo que la vuelve legítima es que sea **declarada**, no una conclusión encubierta. Hoy es encubierta.

1.3 "MTTR real" está mal etiquetado — mide gestión declarada, no remediación verificada

Antipatrón "**MTTR nominal**" (Sección 19): "Llamar MTTR a un intervalo que mide gestión declarada, no remediación verificada. El nombre promete evidencia que el dato no tiene."

`AssetVulnerabilityRepository.findAverageMtttrDays()` (`AssetVulnerabilityRepository.java:74-81`) está comentado explícitamente como "MTTR real: desde la primera detección [...] hasta que un analista lo marcó RESUELTA", y `DashboardService.java:39-40` lo expone al frontend bajo la etiqueta `mttrDays`. Pero el propio marco (Sección 13.2 y 10.1) es preciso: un intervalo que termina en una **declaración humana** (`updateStatus` a `RESUELTA`) es *tiempo de gestión*, no MTTR en sentido estricto — el salto a MTTR real exige que el segundo extremo del intervalo sea una verificación del sistema (nivel 3+), y aquí no lo es (ver 1.2). El dato puede seguir siendo útil, pero necesita el nombre correcto y su ancla declarada (Sección 13.4).

1.4 No hay vocabulario de estados de evidencia (VEX) ni distinción remediar/mitigar/descartar/aceptar

`AssetVulnerability.java:37-42` y `VulnerabilityAuditService.java:42-46` solo contemplan tres estados de triage: `DETECTADA`, `EN_ANALISIS`, `RESUELTA`. Frente al modelo mínimo de la Sección 8.1 (Detectada, Reconocida, En remediación, Remediación declarada, Remediación verificada, **Mitigada**, **No aplica**, **Riesgo aceptado**, Reabierta), faltan:

- **Mitigada**: no hay forma de registrar que el riesgo se redujo por una vía distinta a remediar (control compensatorio, aislamiento) sin marcar el caso como resuelto. Todo lo que no es `RESUELTA` es indistinguible de "sin tratar".
- **No aplica / descartada**, con justificación obligatoria: el campo `decision` (`AssetVulnerability.java:58-59`) es texto libre opcional, no un estado auditable con justificación exigida (Sección 8.2: "un descarte sin justificación registrada no es auditable").
- **Riesgo aceptado**, con responsable y **fecha de caducidad obligatoria**: no existe en absoluto. El marco marca esto como antipatrón directo si falta ("**Aceptación de riesgo perpetua**", Sección 19).

Esto no es un matiz de nomenclatura: sin estos estados, remediar y mitigar quedan colapsados en un único campo "resuelta", perdiendo exactamente la información que la Sección 8.1 dice que se pierde

al colapsarlos.

2. Alta prioridad

2.1 No existe plazo (SLA/due date) derivado de un criterio de priorización (Componente C)

Búsqueda dirigida sobre todo el backend (`SLA`, `deadline`, `due_date`, `KEV`, `EPSS`) no arrojó resultados. `AssetVulnerability` tiene `priority` (denormalizado del hallazgo) pero ningún campo de fecha límite, ni las variables que la producirían. Sin este componente, según la Sección 7, *"no hay cumplimiento que medir: el plazo es un número arbitrario"* — y aquí directamente no hay plazo. Esto bloquea en cascada el KPI "Cumplimiento de SLA", uno de los diez del catálogo mínimo (Sección 13.3) y de la vista ejecutiva recomendada (Sección 14.1).

2.2 El criterio de priorización depende solo de severidad/CVSS, no del modelo de cuatro variables (BOD 26-04)

El campo `priority` se propaga directamente desde el hallazgo del escáner (`applyLifecycle()`, `VulnerabilityAuditService.java:258,271,285`) sin incorporar exposición del activo, estado KEV, ni señales de explotabilidad más allá del booleano `exploited`. El marco (Sección 21.1) es explícito: *"un sistema que en 2026 derive sus plazos exclusivamente de CVSS está aplicando un criterio que la principal autoridad de referencia acaba de abandonar."* No es necesario adoptar la BOD 26-04 completa, pero el diseño actual no deja espacio para incorporar exposición/KEV/EPSS como señales adicionales de priorización.

2.3 No hay escala de evidencia (E0–E6) ni captura de evidencia de respaldo

El campo `decision` (texto libre) es, en el mejor caso, evidencia **E1** (declaración manual). No existe forma de adjuntar ni referenciar capturas, reportes de escáner, commits/PRs, ni de registrar qué nivel de evidencia respalda cada cierre (Sección 8.3). Esto impide distinguir, como pide el marco, una métrica que mide *actividad del proceso* de una que mide *reducción observada de exposición* (Sección 13.1) — hoy todo lo cerrado tiene la misma fuerza probatoria nominal, sea que lo cerró una declaración o una inferencia por ausencia.

2.4 No hay reverificación periódica explícita ni manejo de advisories que cambian

El modelo no contempla el caso en que el advisory de origen amplía su rango afectado, se retracta (`withdrawn`) o se reclasifica después de publicado (Sección 7.1.1, motivo 4, y Sección 10.3). `GhsaAdvisoryCache` (`GhsaAdvisoryCache.java`) solo cachea `summary` / `description` / `referenceUrls` de GitHub — no guarda el rango de versiones afectado, ni si el advisory está `withdrawn`, ni distingue `reviewed` de `unreviewed` (Sección 7.2). Un caso resuelto no vuelve a evaluarse contra un advisory que cambió; solo se reabre si el mismo CVE vuelve a aparecer en un escaneo, lo cual cubre parcialmente la recurrencia pero no la reverificación por cambio de la fuente.

2.5 GHSA se usa solo como texto de referencia, no como fuente estructurada para verificación

Siguiendo el punto anterior: el pipeline de detección real (versión vulnerable vs. rango) ocurre fuera de este repositorio (en n8n, según los comentarios del código); dentro de la aplicación, GHSA es una cadena de texto (`ghsaId`) y un cache de descripción. El modelo de datos no puede hoy registrar la confiabilidad `reviewed` / `unreviewed` de un advisory (Sección 7.2), lo cual importa porque el propio marco señala que confiar en un rango `unreviewed` puede producir falsos cierres — justo el tipo de error que el Componente D debería prevenir.

3. Prioridad media

3.1 KPIs de la tabla de dependencias (Sección 13.2) no calculados

`DashboardService.getStats()` calcula: total de vulnerabilidades, abiertas, críticas, resueltas del mes, `mttrDays` (con la salvedad de 1.3), distribución por severidad/estado y tendencia de 30 días. Frente al catálogo de referencia, **faltan por completo**:

- **MTTD** (publicación del advisory → detección propia) — no hay campo de fecha de publicación del advisory.
- **Tiempo medio de reconocimiento** (detección → primer triage humano) — no existe un timestamp de "reconocida" separado de "en análisis".
- **MTTV** (declaración → verificación) — no existen ambos timestamps por separado (ver 1.3).
- **Tasa de eficacia de remediación** (declaradas vs. verificadas) — requiere justamente los dos timestamps anteriores.
- **Cumplimiento de SLA** — bloqueado por 2.1.
- **Tiempo de permanencia por estado** — bloqueado por 1.1 (sin historial de transiciones no hay duración por estado).
- **Cobertura del escaneo** — bloqueado por 1.2.
- **Curva de aging** — los datos base (`firstDetectedAt`) existen pero no se calcula ni se expone.
- **Tasa de recurrencia** — el dato existe (`reopenCount` , `AssetVulnerability.java:73-74`) pero no se agrega ni se expone en `DashboardStatsDTO` ni en ningún endpoint; es una brecha de "no calculado", no de "no modelado".

3.2 Asignación de responsable sin estructura de rol ni SLA por caso

`assignedTo` (`AssetVulnerability.java:61-62`) es un `String` libre, sin relación a `User` , a un equipo, ni a un SLA propio del caso (Sección 6.4: "*Responsable, Equipo, Fecha límite, SLA*"). El sistema sí tiene control de acceso por alcance (`UserAssetAssignment` , usado en `VulnerabilityAuditService.applyScope*`), lo cual cubre visibilidad, pero no asignación operativa con

trazabilidad de carga de trabajo (KPI "vulnerabilidades por responsable" del catálogo, Sección 13.3 #8, no es calculable de forma confiable sobre texto libre no normalizado).

3.3 Hallazgo persistente y ciclo de remediación no están separados

El modelo de referencia (Sección 16) separa explícitamente el "hallazgo" (persistente, uno por par activo-advisory) del "ciclo de remediación" (uno o más por hallazgo, cada uno con sus propios timestamps). `AssetVulnerability` mezcla ambos: es persistente por par `(softwareComponent, cveId)` — lo cual está bien — pero al reabrirse `(applyLifecycle(),` rama `else` en `VulnerabilityAuditService.java:277-292)` **sobrescribe** `detectionStatus` / `triageStatus` en la misma fila e incrementa `reopenCount`, sin crear un registro de ciclo nuevo con su propio `detected_at` / `resolved_at`. El marco lo anticipa como advertencia de diseño (Sección 11, nota final): *"Un modelo que guarda los timestamps directamente en el hallazgo solo puede representar un ciclo y sobrescribe la historia en cada reapertura [...] migrar después obliga a rehacer todas las consultas de métricas."*

3.4 Identidad del hallazgo sin ubicación (`location_key`)

La identidad hoy es `(softwareComponent_id, cveId)` (`AssetVulnerabilityRepository.findBySoftwareComponent_IdAndCveId`, línea 16). El marco (Sección 7.1) exige un tercer término, la ubicación dentro del activo, *"sin el tercer término, dos ocurrencias distintas de la misma vulnerabilidad en el mismo activo colapsan en una."* En el modelo actual esto es aceptable mientras la granularidad de detección sea "un CVE por componente"; se vuelve una brecha real si en algún momento el mismo CVE puede aplicar a más de una ruta/instancia dentro del mismo componente.

4. Prioridad baja / madurez avanzada

Estos puntos son legítimos según el marco, pero corresponden a un estadio de madurez posterior al que exige resolver primero las secciones 1 y 2 — o dependen de señales externas que hoy no están integradas.

4.1 Sin deduplicación entre múltiples fuentes de detección

El marco señala (Sección 18.2, aporte de DefectDojo) que en cuanto hay más de una herramienta de detección aparece el problema de hallazgos duplicados sobre el mismo par activo-vulnerabilidad. Hoy el sistema tiene una única fuente de detección (el pipeline vía n8n), por lo que esta brecha es hoy hipotética — pero conviene señalarla porque escala mal si se agrega una segunda fuente (por ejemplo, un SAST/DAST directo) sin resolverla primero.

4.2 Sin integración de señales KEV / EPSS / SSVC para plazo dinámico

Relacionado con 2.2, pero en su forma completa (Sección 21.4): ni el catálogo KEV de CISA ni el puntaje EPSS de FIRST están integrados como señales de priorización. Es una mejora de madurez, no un requisito para que el sistema sea "tracking" en sentido estricto — el marco mismo aclara que la BOD 26-04 solo es vinculante para agencias federales de EE. UU.; para esta organización es marco de referencia, no obligación.

4.3 Sin autoevaluación formal contra el modelo de madurez SANS

El programa no ha sido evaluado contra los cinco dominios del Vulnerability Management Maturity Model (Sección 15.1). Es un ejercicio de gestión, no una carencia técnica del sistema, y tiene más sentido hacerlo una vez resueltas las brechas críticas — evaluarlo ahora solo confirmaría lo que ya es evidente por las secciones 1 y 2.

4.4 Sin anclaje explícito al marco regulatorio argentino vigente

El sistema no referencia la Decisión Administrativa 641/2021 ni al Centro Nacional de Ciberseguridad como autoridad de referencia local (Sección 22). No es una carencia funcional del tracking, pero es relevante si el sistema necesita, en algún momento, demostrar alineación normativa ante una auditoría o un cliente del sector público argentino.

5. Fortalezas existentes que conviene preservar

El marco insiste en que las decisiones correctas también merecen quedar documentadas, para no perderlas al priorizar qué falta:

- **La unidad del tracking es el par (componente, CVE), no el CVE global.** `AssetVulnerability` está modelado exactamente como pide la Sección 4: el mismo CVE puede estar `OPEN` en un componente y `RESOLVED` en otro, sin colisión. Esto evita el antipatrón más citado del documento ("El CVE como unidad de cierre").
- **Identidad del inventario correctamente reconciliada.** `InventoryService.process()` (`InventoryService.java:75-100`) actualiza el componente existente por (`assetId`, `software`, `ecosystem`) normalizado desde el `url`, en vez de insertar incondicionalmente, y no depende de atributos volátiles como IP o hostname — justo lo que pide la Sección 7.1 y evita el antipatrón "Identidad volátil".
- **Doble eje de estado (`detectionStatus` vs. `triageStatus`).** El diseño distingue explícitamente "lo que el sistema observa" de "lo que el analista gestiona" (`AssetVulnerability.java:7-19`), que es conceptualmente el mismo principio que separa el flujo operativo del modelo de evidencia en la Sección 6.11 del marco — aunque, como se señala en 1.1 y 1.4, ambos ejes siguen siendo campos mutables sin historial propio.
- **Reapertura automática con contador.** `reopenCount` y la reapertura forzada de `triageStatus` a `DETECTADA` al reaparecer un CVE (`VulnerabilityAuditService.java:277-291`) cubren, aunque sea

de forma parcial (ver 3.3), el requisito central de la Sección 11: que la desaparición y reaparición de un hallazgo no quede invisible para el equipo.